1. What are the main features of Java?

- Platform-independent: Write Once Run Anywhere via JVM and bytecode.
- Object-Oriented: Everything is an object, supporting encapsulation, inheritance, polymorphism, and abstraction.
- Robust: Strong memory management, automatic garbage collection, exception handling.
- Simple: Easy syntax, automatic memory management, no pointers.
- Multithreaded: Supports concurrent programming with threads.
- Secure: Bytecode verifier, sandbox model for applets.
- High Performance: JIT compiler optimizes performance.
- Distributed: Supports networking and web technologies.
- Dynamic: Supports runtime dynamic class loading and reflection.

Follow-up:

- Java is platform-independent due to compiling into bytecode executed by JVM on any OS.
- Memory management is done via heap space and garbage collector which removes unused objects efficiently.

---

2. What are the different class loaders in Java?

- Bootstrap ClassLoader: Loads core Java classes from <JAVA_HOME>/lib.
- Extension ClassLoader: Loads classes from <JAVA_HOME>/lib/ext.
- System/Application ClassLoader: Loads classes from application classpath.
- Custom ClassLoader: User-defined class loaders for dynamic loading.

---

3. What is bytecode in Java, and how is it generated?

- Bytecode is an intermediate, platform-independent set of instructions generated by the Java compiler (javac) from source code. It is stored in .class files.
- Follow-up: JVM interprets bytecode or uses JIT compiler for native execution.

---

4. What is the Just-In-Time (JIT) compiler, and how does it optimize performance?

- JIT compiles bytecode into native machine code at runtime, improving performance by avoiding repeated interpretation.
- Follow-up: JIT compilation happens during runtime, while interpretation executes bytecode instruction-by-instruction.

---

5. How does Java handle memory management for loaded classes?

- Classes are loaded into *method area* by class loaders. Unreferenced classes can get unloaded by JVM if no longer in use.
- Follow-up: Class unloading happens when class loader is dereferenced and garbage collected.

---

6. What is the difference between stack memory and heap memory in Java?

- Stack: Stores method call frames, local variables, and references; fast access; size limited; each thread has its own stack.
- Heap: Stores all Java objects and instance variables; shared among threads; subject to garbage collection.

---

7. How does the stack memory work during method calls?

- Each method call creates a *stack frame* with local variables and return address.
- Follow-up: Upon method return, its stack frame is popped and local variables go out of scope.

---

8. What are the implications of memory allocation in the heap for object creation?

- Objects are created in heap and heap space is managed dynamically.
- Follow-up: Garbage collection frees unused objects, preventing memory leaks.

---

9. How does the JVM manage memory for objects in the heap?

- JVM uses generational heap structure, tracking *young*, *old*, and *permanent* generations to optimize garbage collection.
- Follow-up: Garbage collector identifies unreachable objects and reclaims memory.

---

10. What are stack overflow and heap overflow errors?

- Stack Overflow: When the call stack exceeds its limit, often due to deep or infinite recursion.
- Heap Overflow: When JVM cannot allocate memory on the heap for new objects.
- Prevent: Avoid infinite recursion; tune JVM memory options; use efficient data structures.

---

11. Explain the concept of OOP in Java.

- OOP is a programming paradigm based on *objects* which contain data and behavior. Java supports OOP principles:
    - Encapsulation: Hide internal object details with access specifiers.
    - Inheritance: Reuse code via parent-child class relationship.
    - Polymorphism: Ability to take many forms via method overriding/overloading.
    - Abstraction: Hide complexity, show essential features using abstract classes/interfaces.

---

12. What is the difference between == and equals() in Java?

- == checks reference equality (whether two references point to the same object).
- equals() checks logical equality (whether two objects are meaningfully equal).
- Scenario:
  Using == on two different string objects with the same content returns false; using equals() returns true.
- equals() is often overridden in user classes to compare fields.

---

13. What is the significance of the final keyword in Java?

- final variable: Immutable once initialized.
- final method: Cannot be overridden.
- final class: Cannot be subclassed. Used to prevent inheritance/modification for security or design reasons.

---

14. Explain the difference between ArrayList and LinkedList.

- ArrayList: Backed by dynamic array; fast random access O(1); slow insert/delete O(n) due to shifting.
- LinkedList: Doubly-linked list; slow access O(n); fast insertion/deletion at ends or iterator O(1).
- Use ArrayList when frequent access; LinkedList when frequent insertions/deletions at ends.

---

15. What is the Java memory model?

- Defines how Java manages and accesses memory in multithreaded environments to ensure visibility and ordering consistency of variables across threads.

---

16. Explain how the try-catch-finally block works in exception handling.

- try: Code that may throw exceptions.
- catch: Handle exceptions.
- finally: Runs regardless of exceptions for cleanup.
- If exception occurs in finally, it can suppress original exceptions.
- Example where finally doesn’t execute: If JVM exits or thread is killed.

---

17. What are checked and unchecked exceptions in Java?

- Checked exceptions: Must be declared or handled (e.g., IOException).
- Unchecked exceptions: Runtime exceptions not required to be declared (e.g., NullPointerException).
- You can catch both in a single try-catch by catching Exception.

---

18. What is the difference between an interface and an abstract class in Java?

- Interface: All abstract methods (until Java 8+), multiple inheritance allowed. Used for capability declaration.
- Abstract class: Can have concrete & abstract methods, single inheritance, used for share common code.
- Abstract class can implement interfaces.

---

19. What is the significance of the volatile keyword in Java?

- Ensures visibility of changes to variables across threads, preventing caching issues.
- Prevents instruction reordering optimizations on volatile variables.

---

20. How does volatile differ from synchronized?

- volatile is lighter-weight, providing visibility but no atomicity.
- synchronized enforces mutual exclusion and visibility, guaranteeing atomic access.

21. What is the use of the transient keyword in Java?

Answer: The transient keyword marks a variable to be skipped during serialization. Transient fields are not inherited by subclasses because serialization applies to the class defining the field. If a transient variable is serialized, its value is not saved and defaults after deserialization.

22. What are wrapper classes in Java?

Answer: Wrapper classes provide object representations for primitive types (e.g., Integer for int). Autoboxing/unboxing automatically converts between primitives and wrappers but can introduce performance overhead due to object creation.

23. What is the significance of the this keyword in Java?

Answer: The this keyword refers to the current object instance. You cannot assign a value to this inside a method since it is final. Passing this to a static method is allowed, but the static method can't access instance members through it.

24. Explain multithreading in Java.

Answer: Multithreading allows concurrent execution of threads within a program. Thread class represents a thread; Runnable is a task definition. Thread safety can be ensured using synchronization or concurrent utilities.

25. What is the difference between String, StringBuilder, and StringBuffer?

Answer: String is immutable; StringBuilder and StringBuffer are mutable. StringBuilder is faster but not thread-safe, while StringBuffer is synchronized and safe in multi-threaded environments.

26. What is the purpose of the synchronized keyword in Java?

Answer: synchronized ensures mutual exclusion for methods or blocks. Static synchronized methods lock on the class object. Excessive use can lead to performance bottlenecks.

27. What is the Java Collections Framework?

Answer: It is a set of interfaces and classes for storing and manipulating groups of objects. Key interfaces include List, Set, and Map. HashMap ensures unique keys via hashCode() and equals() methods.

28. What is garbage collection in Java?

Answer: Garbage collection automatically frees memory of unreferenced objects. Common collectors include Serial, Parallel, CMS, and G1. You cannot force GC; System.gc() is only a suggestion.

29. What are lambda expressions in Java?

Answer: Lambda expressions enable inline implementation of functional interfaces, simplifying functional programming. Example: (a, b) -> a + b for a comparator.

30. What is the use of Optional in Java 8?

Answer: Optional helps avoid NullPointerException by explicitly representing nullable values. Optional.of() rejects nulls; Optional.ofNullable() allows null values safely.

31. How does a HashMap work internally?

Answer: HashMap uses an array of buckets, where keys map to bucket indices via hashing. Collisions are handled by linked lists or balanced trees (in Java 8+). Load factor controls resize thresholds.

32. What is a thread pool, and why is it used?

Answer: A thread pool manages reusable threads for efficient execution. newCachedThreadPool() creates threads as needed; newFixedThreadPool() has a fixed number. It reduces overhead of creating and destroying threads.

33. What are generics in Java?

Answer: Generics provide type safety by allowing parameterized types, improving code quality and compile-time checking. Primitive types cannot be used directly in generics.

34. Explain the concept of class loading in Java.

Answer: Class loaders load classes into JVM. Types include Bootstrap, Extension, and System loaders. Classes with the same name but different loaders are treated as distinct.

35. What are the differences between Comparator and Comparable in Java?

Answer: Comparable defines natural ordering via compareTo(). Comparator defines custom orderings. Use Comparator for multiple sorting strategies. Equal sorting keys keep original order.

36. What is the purpose of super in Java?

Answer: super accesses superclass methods or constructors. Example: super() calls parent constructor. In method overriding, super invokes the parent method.

37. Explain the Singleton design pattern in Java.

Answer: Singleton ensures a class has only one instance. Thread-safe implementations use synchronization or double-checked locking. Singletons can cause issues in distributed systems due to multiple class loaders or serialization.

38. What are method references in Java 8?

Answer: Method references are shorthand expressions pointing to methods or constructors, often more concise than lambda expressions. Types include static methods, instance methods, and constructor references.

39. What is the default keyword in Java interfaces?

Answer: The default keyword allows method implementations in interfaces (introduced in Java 8). Default methods can be overridden for backward compatibility.

40. What is a ConcurrentHashMap, and how is it different from a HashMap?

Answer: ConcurrentHashMap is thread-safe with lock-striping, allowing higher concurrency and better performance than synchronized HashMap in multi-threaded contexts.

41. What is the difference between the wait() and sleep() methods in Java?

Answer: wait() releases the lock and must be called inside synchronized blocks; sleep() does not release the lock and can be called anywhere. Sleeping threads can be interrupted; waiting threads are notified.

42. Explain how notify() and notifyAll() work in Java.

Answer: notify() wakes one waiting thread; notifyAll() wakes all waiting threads. Use notifyAll() to avoid deadlocks. Calling notify() without owning the monitor throws IllegalMonitorStateException.

43. What is reflection in Java?

Answer: Reflection allows runtime inspection and modification of classes, methods, and fields. Used in frameworks like Spring for dependency injection. Reflection has performance overhead and security risks.

44. Explain the difference between Serializable and Externalizable.

Answer: Serializable is a marker interface for default serialization. Externalizable requires explicit implementation of writeExternal and readExternal for custom serialization control.

45. What is the purpose of the enum keyword in Java?

Answer: enum defines a fixed set of constants as a type-safe enumeration. Enums cannot be extended but can have fields and methods.

46. What is a deadlock in Java?

Answer: Deadlock occurs when threads wait indefinitely for locks held by each other. Prevent by avoiding circular waits, using lock ordering, or timeouts.

47. Explain the concept of immutability in Java with an example.

Answer: Immutability means an object's state cannot be changed after creation (e.g., String). Immutability improves thread safety and performance. To create immutable classes, use final fields, no setters, and careful construction.

48. What is the difference between throw and throws in Java?

Answer: throw is used to explicitly throw an exception. throws declares exceptions a method might throw. Methods can throw multiple exceptions; declaring exceptions not thrown is legal but discouraged.

49. What are functional interfaces in Java?

Answer: Functional interfaces have exactly one abstract method, enabling use with lambda expressions. Common examples: Runnable, Callable, Comparator.

50. What is the difference between String and char[] for storing passwords in Java?

Answer: char[] is preferred over String for passwords because it can be cleared explicitly from memory, whereas String remains in memory until GC. Use Arrays.fill(char[], '\0') to clear securely.

Set-2

1. What is the difference between shallow cloning and deep cloning in Java?
   Shallow cloning copies the object's fields as references, so objects inside the clone share the same references. Deep cloning copies all objects recursively, creating independent copies. Shallow cloning uses Object.clone() method; deep cloning requires custom implementation.

2. What are the key differences between a HashSet and a TreeSet?
   HashSet stores elements in a hash table, offers constant-time performance, and doesn't maintain order. TreeSet stores elements in a red-black tree, maintains natural ordering, and has logarithmic time operations. Use TreeSet when sorting is needed.

3. What is method hiding in Java?
   Method hiding occurs when a subclass defines a static method with the same signature as a static method in the superclass. It differs from overriding because static methods are bound at compile-time. Method hiding can lead to unexpected behavior if accessed via superclass reference.

4. What are the differences between try-with-resources and a traditional try-catch-finally block?
   Try-with-resources automatically closes resources that implement AutoCloseable at the end of the block, ensuring proper cleanup. Traditional try-catch-finally requires explicit resource closure, which is prone to errors.

5. What is the difference between Callable and Runnable in Java?
   Runnable does not return a result and cannot throw checked exceptions. Callable returns a result and can throw checked exceptions. Callable can be used with Future to retrieve computation results asynchronously.

6. What is the difference between checked and unchecked exceptions in Java?
   Checked exceptions must be either caught or declared in the method signature. Unchecked exceptions (subclasses of RuntimeException) do not require handling. Checked exceptions represent foreseeable errors; unchecked exceptions usually indicate programming bugs.

7. Explain the use of try, catch, and finally blocks in Java.
   try defines a block that may throw exceptions. catch handles exceptions. finally contains code that always runs after try/catch, regardless of exceptions. The finally block can be omitted but is useful for cleanup.

8. What is the purpose of the throws keyword in method declarations?
   throws declares exceptions that a method may throw, informing callers to handle them. Methods can throw both checked and unchecked exceptions. Throws affects method signature, enabling compile-time checking.

9. How would you create a custom exception in Java?
   Create by extending Exception (checked) or RuntimeException (unchecked). Include constructors and optionally extra fields or methods for additional error information.

10. What is the difference between throw and throws in Java?
    throw is used to explicitly throw an exception instance. throws declares exceptions that a method might throw.

11. What is the purpose of the try-with-resources statement?
    It automatically closes resources implementing AutoCloseable after use, ensuring proper cleanup and avoiding resource leaks. Custom classes can implement AutoCloseable to work with try-with-resources.

12. What are some best practices for exception handling in Java?
    Avoid catching Exception or Throwable broadly; catch specific exceptions. Log exceptions properly, and avoid suppressing original exceptions. Use checked exceptions for recoverable conditions.

13. Explain the finally block. Can it be skipped?
    Finally always executes after try/catch, except when JVM exits or thread is killed. Returns inside try/catch do not skip finally. Exceptions in finally can suppress original exceptions.

14. Difference between re-throwing and throwing a new exception?
    Re-throwing preserves original stack trace; throwing a new exception replaces it. Wrapping exceptions is used for abstraction or adding context.

15. Difference between final, finally, and finalize() in Java?

- final: keyword to define constants or prevent modification.
- finally: block for cleanup in exception handling.
- finalize(): deprecated method called by garbage collector; use try-with-resources or cleaners instead.

16. What is a constructor in Java?
    A special method to initialize new objects. Constructors cannot be final, static, or abstract because they are not inherited and do not return values.

17. Difference between default and parameterized constructor?
    Default constructor has no parameters and sets default values; parameterized constructor accepts arguments to initialize fields. If no constructor is defined, compiler inserts a default.

18. Can a constructor call another constructor in the same class?
    Yes, using this() syntax (constructor chaining). It must be the first statement.

19. Can a constructor throw an exception?
    Yes, constructors can throw exceptions to indicate initialization failure. Best handled with care, as failed construction leaves no object.

20. What is a static constructor (static block)?
    Static block initializes static variables; executed once at class loading. Static blocks cannot access instance variables.

21. Difference between a constructor and a method?
    Constructor initializes objects, has no return type, and name matches class. Methods perform actions, have return types, and can be called anytime.

22. Purpose of the static keyword?
    Defines class-level variables/methods shared by all instances. Static methods cannot access instance variables directly.

23. Explain the main method in Java. Why public, static, void?
    Entry point of Java applications. public allows JVM access, static allows JVM to call without instance, void means it returns nothing.

24. Difference between static and instance methods?
    Static belongs to class; instance belongs to object. Static methods cannot be overridden, only hidden.

25. Explain memory management in Java.
    Java manages memory in stack (method calls/local vars) and heap (objects). Garbage collector automatically frees unused heap memory.

26. What is the garbage collector?
    Background process reclaiming memory from unreachable objects. Algorithms include Mark & Sweep, G1, CMS.

27. Ways to prevent garbage collection?
    Strong references keep objects alive. Overusing finalize() cannot prevent collection reliably.

28. Role of finalize() method?
    Deprecated cleanup method called before garbage collection. Use try-with-resources or cleaners instead.

29. How is Object class the superclass of all classes?
    All classes implicitly extend Object, inheriting methods like toString(), equals(), and hashCode().

30. Commonly used Object class methods?
    equals(), hashCode(), and toString(); override them to customize equality and string representation.

31. Four main OOP principles?
    Encapsulation, Inheritance, Polymorphism, Abstraction.

32. Explain encapsulation.
    Hiding internal state using private/protected access, exposing via getters/setters to protect data integrity.

33. What is inheritance?
    Mechanism to derive new classes from existing ones, promoting code reuse.

34. Explain polymorphism with example.
    Ability to reference different object types via a superclass/interface type; e.g., overriding methods behave differently at runtime.

35. What is abstraction?
    Hiding implementation details, providing only essential features via abstract classes or interfaces.

36. Method overriding?
    Subclass provides specific implementation of superclass method to enable runtime polymorphism.

37. Method overloading?
    Multiple methods in same class with same name but different parameters.

38. Difference between abstract class and interface?
    Abstract class can have state and constructors; interface cannot (before Java 8). Interfaces support multiple inheritance.

39. Constructors and inheritance?
    Subclasses call superclass constructors with super(); if omitted, default constructor called automatically.

40. Relationship between objects and classes?
    Classes are blueprints; objects are instances created using new.

41. Difference between this and super?
    this refers to current object; super refers to superclass members.

42. Difference between composition and inheritance?
    Inheritance is "is-a" relationship; composition is "has-a." Prefer composition to reduce tight coupling.

43. Significance of instanceof?
    Checks if an object is an instance of a class/interface, supporting safe downcasting.

44. Why no multiple inheritance with classes but allowed with interfaces?
    To avoid Diamond problem; interfaces with default methods complicate it but are resolved with explicit overrides.

45. Getter and setter methods?
    Accessors and mutators that promote encapsulation by controlling access to fields.

46. What is an object?
    Runtime instance of a class with identity, state, and behavior.

47. Explain upcasting and downcasting.
    Upcasting: Subclass to superclass casting (safe).
    Downcasting: Superclass to subclass casting (requires checking, can throw ClassCastException).

48. What is toString() method?
    Returns string representation of the object. Overriding improves clarity and debugging.

49. Inner classes and types?
    Classes defined within other classes: non-static inner, static nested, local, and anonymous.

50. Difference between single and multilevel inheritance?
    Single: subclass inherits one superclass.
    Multilevel: subclass inherits from a subclass, forming inheritance chain.

Set-3

1. How does Java handle access control in inheritance (private, protected, public)?
   Private members are not inherited and accessible only within the class. Protected members are accessible in the same package and subclasses. Public members are accessible everywhere.
2. Can you inherit private fields and methods from a superclass? How can a subclass access them?
   Private members are not inherited directly but can be accessed via public or protected getter/setter methods provided in the superclass.
3. Explain the "is-a" relationship in inheritance with an example.
   An "is-a" relationship indicates subclassing (e.g., Dog "is-a" Animal). It differs from "has-a" (composition). If subclass adds unrelated behaviors, "is-a" relation may break logically.
4. What is the role of polymorphism in collections like List, Set in Java?
   Polymorphism allows storing objects of different subclasses under a collection of superclass type, enabling flexible and dynamic method calls.
5. What happens if you store subclass objects in a collection of the superclass type?
   It is allowed; the collection handles superclass references, enabling polymorphic behavior on stored objects.
6. Can an abstract class have a constructor?
   Yes, abstract classes can have constructors called from subclasses to initialize inherited state.
7. What is the difference between an abstract method and a concrete method?
   Abstract methods have no body and must be overridden. Concrete methods have implementations.
8. Can you provide an example where an interface is more appropriate than an abstract class?
   When multiple inheritance of behavior contracts is needed, interfaces are preferable as Java supports multiple interfaces but single class inheritance.
9. What are the limitations of abstraction in Java?
   Abstraction cannot hide all implementation details; sometimes exposing APIs is necessary. Also, abstraction can introduce performance overhead.
10. How does abstraction differ between abstract classes and interfaces?
    Abstract classes can have state and method implementations; interfaces (Java 8+) can have default methods but generally define contracts.

11. How does Java's private access modifier help in encapsulation?
    It restricts direct access to class members, protecting internal data from unauthorized external access.

12. Is multiple inheritance possible in Java with classes?
    No, but multiple inheritance via interfaces is possible, avoiding the diamond problem.

13. Can a class implement multiple interfaces?
    Yes, a class can implement multiple interfaces, but must handle method signature conflicts, especially with default methods.

14. Why can't we instantiate an abstract class?
    Because it may have incomplete implementations; only subclasses with full implementations can be instantiated.

15. Why is Java's Object class considered the superclass of all classes?
    It provides basic methods (toString(), equals(), hashCode()) for every Java object.

16. What is method hiding and its relation to static methods?
    Static methods are resolved at compile time and can be hidden, not overridden.

17. Explain the architecture of the Java Collections Framework.
    Includes interfaces like Collection, List, Set, Map and their implementations like ArrayList, HashSet, HashMap.

18. Difference between Collection and Collections classes?
    Collection is a root interface; Collections is a utility class with static methods for collection operations.

19. Root interface of Collections Framework and its subinterfaces?
    Collection is root; subinterfaces include List, Set, and Queue.

20. Difference between List, Set, and Map?
    List allows duplicates and ordered elements; Set enforces uniqueness; Map stores key-value pairs.

21. How does Collections Framework support generics?
    Enables type-safe operations, catching errors at compile time.

22. Difference between array and collection?
    Arrays have fixed size and can store primitives; collections are dynamic and store objects.

23. Difference between ArrayList and LinkedList?
    ArrayList is backed by dynamic arrays, better for random access; LinkedList uses nodes, better for frequent insertions/deletions.

24. What is a Set and how does it differ from a List?
    Set stores unique elements; List allows duplicates and ordered elements.

25. How does HashSet handle duplicates?
    It uses hashCode() and equals() to ensure uniqueness.

26. Difference between HashMap, TreeMap, LinkedHashMap?
    HashMap unordered; TreeMap sorted by keys; LinkedHashMap maintains insertion order.

27. What is a Queue and how is it different from List?
    Queue enables FIFO operations; List allows random access.

28. How does PriorityQueue work?
    Orders elements based on natural ordering or a Comparator.

29. Difference between Iterator and ListIterator?
    ListIterator supports bidirectional traversal; Iterator supports forward only.

30. Difference between Enumeration and Iterator?
    Enumeration is legacy; Iterator has remove() method.

31. How does HashMap handle collisions?
    Uses chaining with linked lists or balanced trees.

32. Difference between ArrayDeque and LinkedList as Queue?
    ArrayDeque is faster with no capacity restrictions, preferred for queue operations.

33. Key differences between HashMap and Hashtable?
    Hashtable is synchronized; HashMap is not.

34. How does TreeMap sort its keys?
    By natural ordering or Comparator; null keys not permitted.

35. Difference between poll(), remove(), and peek() in Queue?
    poll() returns null if empty; remove() throws exception; peek() returns but does not remove.

36. How does Java Collections provide thread-safety?
    Through synchronized wrappers or concurrent collections like ConcurrentHashMap.

37. Explain fail-fast and fail-safe iterators?
    Fail-fast throw exception on concurrent modification; fail-safe work on a copy.

38. Internal working of HashMap?
    Stores key-value pairs in buckets; uses hashCode and equals for key equality.

39. Importance of overriding hashCode() and equals()?
    Ensures correct key behavior in hash-based collections.

40. What is load factor in HashMap?
    Controls resizing; default 0.75 balances time and space.

41. Differences between HashMap, LinkedHashMap, TreeMap in performance?
    HashMap fastest; LinkedHashMap maintains order; TreeMap slower due to sorting.

42. Can HashMap have null keys and values?
    Yes, one null key allowed; null values allowed.

43. Difference between thread and process in Java?
    Processes have separate memory; threads share memory.

44. Real-life example of multithreading?
    Web server handling multiple client requests concurrently.

45. How does thread communication work?
    Using wait(), notify(), and notifyAll() methods on shared objects.

46. Differences between Thread and Runnable?
    Runnable is a task; Thread executes Runnable. Runnable preferred for multiple inheritance.

47. Explain thread pooling?
    Reuse fixed number of threads to limit overhead.

48. What is a deadlock? How to avoid it?
    Mutual waiting of threads for locks; avoided by lock ordering and timeout.

49. How to handle uncaught exceptions in threads?
    Using UncaughtExceptionHandler interface.

50. What is a generic class and why use it?
    Class with type parameters to enable type safety and reusability.

51. Explain how to define a generic method in Java.
    Generic methods declare their own type parameters, allowing them to operate on different types independently of generic classes.

52. What are wildcards in Java generics? When use ? extends T vs ? super T?
    ? extends T is an upper-bounded wildcard (covariant, read-only), ? super T is a lower-bounded wildcard (contravariant, write-only). Use extends for flexibility in reading, super for flexibility in writing.

53. What are the two main types of streams in Java?
    Byte streams (InputStream/OutputStream) for binary data and character streams (Reader/Writer) for text data.

54. How do you read/write data using FileInputStream/FileOutputStream?
    They handle raw byte streams, used for binary data; reading/writing is done in byte arrays or single bytes.

55. Differences between BufferedReader and FileReader?
    BufferedReader buffers input, improving efficiency, especially for reading characters, lines; FileReader reads characters directly without buffering.

56. Explain working of InputStream and OutputStream.
    Abstract classes representing input and output of byte streams; concrete subclasses handle specific sources/destinations.

57. How to properly close a file resource in Java?
    Use try-with-resources statement to automatically close resources or close explicitly in finally block.

58. How to read a text file using BufferedReader?
    By wrapping a Reader (e.g., FileReader) with BufferedReader to efficiently read text lines.

59. How to write to a file using BufferedWriter?
    Wrap a Writer (e.g., FileWriter) in BufferedWriter; handle exceptions properly; flush and close streams.

60. Difference between PrintWriter and BufferedWriter?
    PrintWriter provides convenient methods like println(); BufferedWriter buffers characters, improving I/O efficiency.

61. How to append data to existing file in Java?
    Pass true as second argument to FileWriter constructor to open file in append mode.

62. Common exceptions during file handling?
    FileNotFoundException, IOException, manage by try-catch and resource management.

63. What is serialization in Java? Why use it?
    Serialization converts objects to byte streams for storage or transmission; Serializable interface marks classes as serializable.

64. How does deserialization work?
    Reads byte stream to reconstruct objects; class must implement Serializable, else NotSerializableException thrown.

65. What is a lambda expression?
    A concise expression to implement functional interfaces, enabling functional programming style.

66. How to define lambda for Comparator in Java?
    Example: (a, b) -> a.compareTo(b)

67. Limitations of lambda expressions?
    Cannot define constructors; no checked exceptions handling; cannot change non-final variables from enclosing scope.

68. How does type inference work with lambdas?
    Compiler infers parameter types from context; explicit types are optional but can be specified.

69. What are functional interfaces?
    Interfaces with a single abstract method; basis for lambda expressions.

70. What is a Stream in Java 8?
    A sequence of elements supporting aggregate operations, different from collections by enabling functional-style operations.

71. Common terminal operations in streams?
    reduce(), collect(), forEach(). Terminal operations mark the end of stream processing.

72. How does map() function work in Streams?
    Transforms elements by applying a function, creating a new stream with mapped elements.

73. How is reduce() used in Streams API?
    Combines stream elements into a single result, like summing numbers.

74. Purpose of default methods in interfaces?
    Provide method implementations in interfaces for backward compatibility and code reuse.

75. Difference between default and static methods in interfaces?
    Default methods belong to instances; static methods belong to the interface class and are called with interface name.

76. Why were default methods introduced in Java 8?
    Allow adding methods to interfaces without breaking existing implementations.

77. How do static methods in interfaces differ from static methods in classes?
    Static interface methods can't be inherited by implementing classes; called using interface name directly.

78. Can an interface have both default and abstract methods?
    Yes; implementing classes must implement abstract methods but can override default methods.

79. What is Optional class and why used?
    Wrapper to represent optional values, helping avoid NullPointerException.

80. How to create an empty Optional?
    Using Optional.empty(). Difference: Optional.of() doesn't accept null; Optional.empty() returns empty Optional.